# PART 5 - MODULES AND FUNCTIONS



- DIFFERENTIAL EQUATIONS
- SOME EXAMPLES
- PRACTICAL 5



chaos....

David Topping: david.topping@manchester.ac.uk

'Vanilla' Python

Data types
Basic operations

Loops
Conditionals

Pandas and data discovery

Geospatial analysis

Statistical Analysis

Cluster Analysis

1 2 3 4 5 6 7 8 9

The Numpy module

Conway's Game Of Life

Differential Equations

Modules and Functions

Creating a notebook assessment
select topic from weeks 4 - 9

10 11 12

# COURSE LAYOUT >>>>

LECTURE + PRACTICAL. SOLUTIONS ARE GIVEN

# RECAP

- Functions help us repeat a set of operations many times
- They are specified using ‘def’ and ‘return’ statements

```
In [ ]: # Lets define a function called 'test'. This function will simply write a message to the screen.
def test():
    print("All work and no play makes Jack a dull boy")
    return

# Call the function
test()
```

- Modules contain functions we can access using the ‘.’ operator

```
In [ ]: import numpy as np
import pandas as pd
import sklearn #the name of the module is different from the official title
import matplotlib
```

- The Numpy module is the standard way to deal with numerical operations in Python

```
In [2]: import numpy as np

array1 = np.array([10,20,30,40,50], dtype=int)
array2 = np.array([0,1,2,3,4], dtype=int)
array3 = np.multiply(array1,array2)

print("array3 = ", array3)
```

```
array3 =  [  0  20  60 120 200]
```

# DIFFERENTIAL EQUATIONS

REAL WORLD



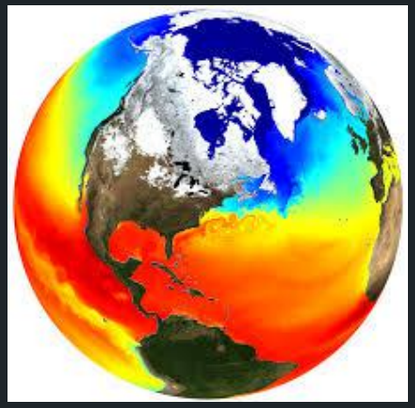
THEORY

$$\frac{dX}{dt} = \sigma(Y - X)$$
$$\frac{dY}{dt} = (\rho - Z)X - Y$$
$$\frac{dZ}{dt} = XY - \beta Z$$

SIMULATE



MODEL WORLD



PROGRAMMING

# DIFFERENTIAL EQUATIONS

INTEGRATION

$$\frac{dF(x)}{dx} = f(x) \qquad F(x) = \int f(x)\,dx$$

DIFFERENTIATION

## SOMETIMES, WE CAN INTEGRATE BY HAND

$$\frac{dN}{dt} = -\lambda N$$

$$\int_{N_o}^{N} \frac{1}{N} dN = \int_{o}^{t} -\lambda\, dt$$

$$\ln\left(\frac{N}{N_o}\right) = -\lambda t$$

$$N = N_o \exp(-\lambda t)!!$$

## SOMETIMES, WE CANT

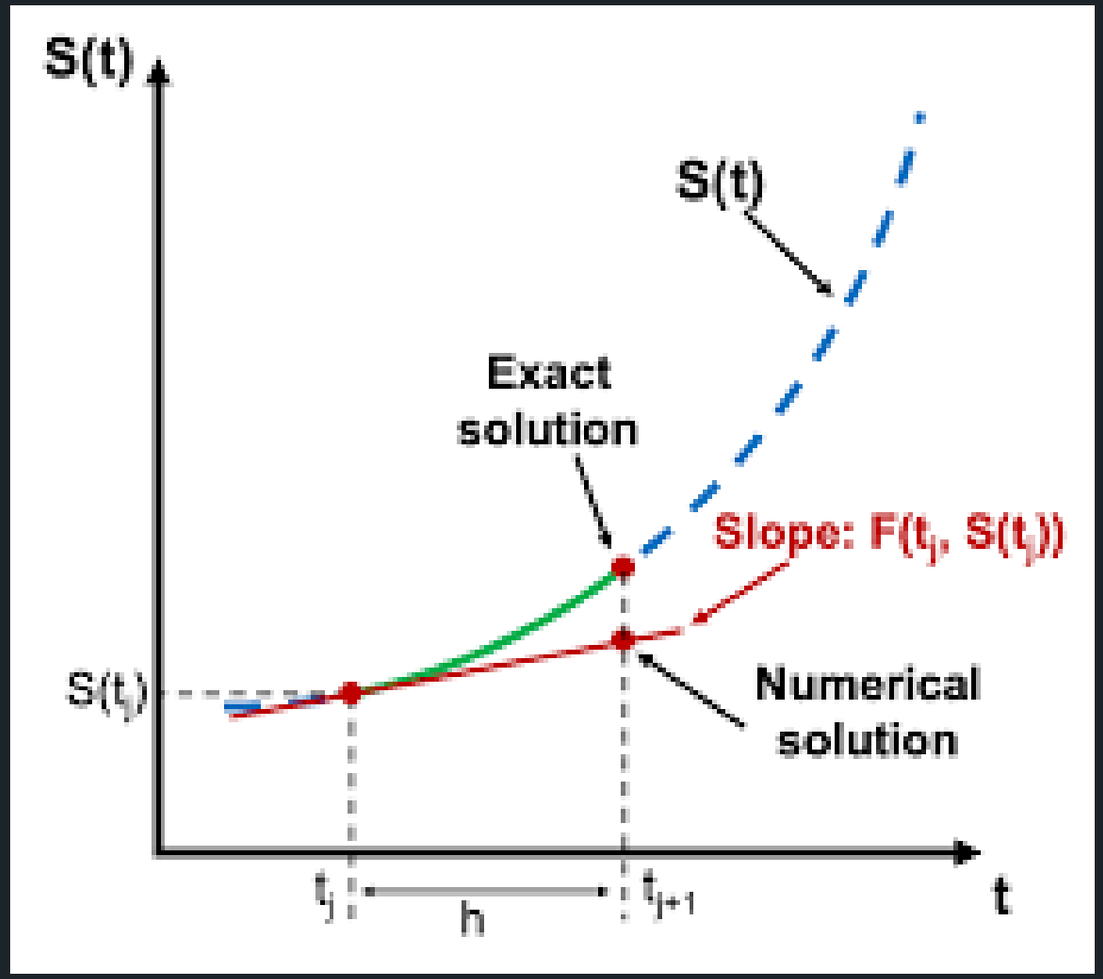
$$\frac{d[A]}{dt} = -k_1[A][X]$$

$$\frac{d[X]}{dt} = k_1[A][X] - k_2[X][Y]$$

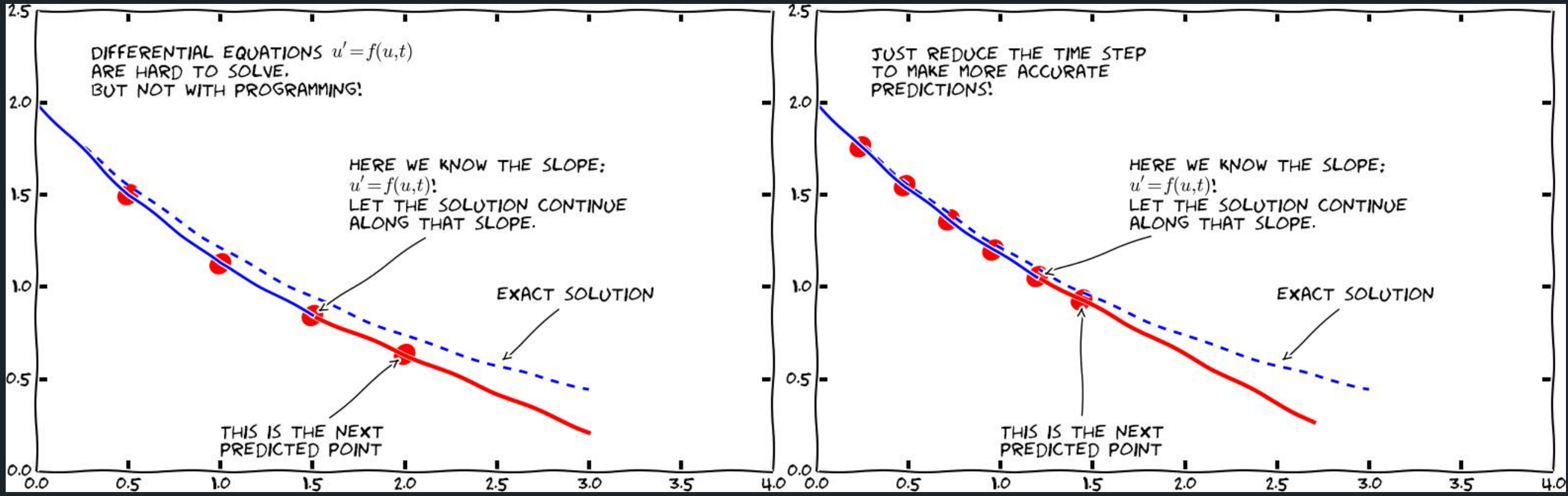
$$\frac{d[Y]}{dt} = k_2[X][Y] - k_3[Y]$$

$$\frac{d[B]}{dt} = k_3[Y]$$

# DIFFERENTIAL EQUATIONS



# DIFFERENTIAL EQUATIONS



# DIFFERENTIAL EQUATIONS



# DIFFERENTIAL EQUATIONS

THERE ARE MANY ODE SOLVERS THAT CAN INTEGRATE COUPLED DIFFERENTIAL EQUATIONS

$$\frac{d[A]}{dt} = -k_1[A][X]$$
$$\frac{d[X]}{dt} = k_1[A][X] - k_2[X][Y]$$
$$\frac{d[Y]}{dt} = k_2[X][Y] - k_3[Y]$$
$$\frac{d[B]}{dt} = k_3[Y]$$

GET PYTHON TO SOLVE IT!

THERE ARE ‘GOOD’ DEFAULT OPTIONS, BUT YOU NEED TO KNOW A LITTLE ABOUT THE PROBLEM YOU

| Solver | Accuracy | Description |
|---|---|---|
| `ode45` | Medium | This should be the first solver you try |
| `ode23` | Low | Less accurate than `ode45` |
| `ode113` | Low to high | For computationally intensive problems |
| `ode15s` | Low to medium | Use if `ode45` fails because the problem is stiff* |

← Runge-Kutta (4,5) formula

CODE DESIGN INCLUDES DOMAIN KNOWLEDGE

# DIFFERENTIAL EQUATIONS

## Comparison Of Differential Equation Solver Software

| Subject/Item | MATLAB | SciPy | deSolve | DifferentialEquations.jl | Sundials | Hairer | ODEPACK/Netlib /NAG | JitCODE | PyDSTool | FATODE | GSL | BOOST | Mathematica | Maple |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Language | MATLAB | Python | R | Julia | C++ and Fortran | Fortran | Fortran | Python | Python | Fortran | C | C++ | Mathematica | Maple |
| Selection of Methods for ODEs | Fair | Poor | Fair | Excellent | Good | Fair | Good | Poor | Poor | Fair | Poor | Fair | Fair | Fair |
| Efficiency* | Poor | Poor**** | Poor*** | Excellent | Excellent | Good | Good | Good | Good | Good | Fair | Fair | Fair | Good |
| Tweakability | Fair | Poor | Good | Excellent | Excellent | Good | Good | Fair | Fair | Fair | Fair | Fair | Good | Fair |
| Event Handling | Good | Good | Fair | Excellent | Good** | None | Good** | None | Fair | None | None | None | Good | Good |
| Symbolic Calculation of Jacobians and Autodifferentiation | None | None | None | Excellent | None | None | None | None | None | None | None | None | Excellent | Excellent |
| Complex Numbers | Excellent | Good | Fair | Good | None | None | None | None | None | None | None | Good | Excellent | Excellent |
| Arbitrary Precision Numbers | None | None | None | Excellent | None | None | None | None | None | None | None | Excellent | Excellent | Excellent |
| Control Over Linear/Nonlinear Solvers | None | Poor | None | Excellent | Excellent | Good | Depends on the solver | None | None | None | None | None | Fair | None |
| Built-in Parallelism | None | None | None | Excellent | Excellent | None | None | None | None | None | None | Fair | None | None |
| Differential-Algebraic Equation (DAE) Solvers | Good | None | Good | Excellent | Good | Excellent | Good | None | Fair | Good | None | None | Good | Good |
| Implicitly-Defined DAE Solvers | Good | None | Excellent | Fair | Excellent | None | Excellent | None | None | None | None | None | Good | None |
| Constant-Lag Delay Differential Equation (DDE) Solvers | Fair | None | Poor | Excellent | None | Good | Fair (via DDVERK) | Fair | None | None | None | None | Good | Excellent |
| State-Dependent DDE Solvers | Poor | None | Poor | Excellent | None | Excellent | Good | None | None | None | None | None | None | Excellent |
| Stochastic Differential Equation (SDE) Solvers | Poor | None | None | Excellent | None | None | None | Good | None | None | None | None | Fair | Poor |
| Specialized Methods for 2nd Order ODEs and Hamiltonians (and Symplectic Integrators) | None | None | None | Excellent | None | Good | None | None | None | None | None | Fair | Good | None |
| Boundary Value Problem (BVP) Solvers | Good | Fair | None | Good | None | None | Good | None | None | None | None | None | Good | Fair |
| GPU Compatibility | None | None | None | Excellent | Good | None | None | None | None | None | None | Good | None | None |
| Analysis Addons (Sensitivity Analysis, Parameter Estimation, etc.) | None | None | None | Excellent | Excellent | None | Good (for some methods like DASPK) | None | Poor | Good | None | None | Excellent | None |

* Efficiency takes into account not only the efficiency of the implementation, but the features of the implemented methods (advanced timestepping controls, existence of methods which are known to be more efficient, Jacobian handling)

# DIFFERENTIAL EQUATIONS

## Solving initial value problems for ODE systems

The solvers are implemented as individual classes, which can be used directly (low-level usage) or through a convenience function.

| | |
|---|---|
| **solve_ivp**(fun, t_span, y0[, method, t_eval, ...]) | Solve an initial value problem for a system of ODEs. |
| **RK23**(fun, t0, y0, t_bound[, max_step, rtol, ...]) | Explicit Runge-Kutta method of order 3(2). |
| **RK45**(fun, t0, y0, t_bound[, max_step, rtol, ...]) | Explicit Runge-Kutta method of order 5(4). |
| **DOP853**(fun, t0, y0, t_bound[, max_step, ...]) | Explicit Runge-Kutta method of order 8. |
| **Radau**(fun, t0, y0, t_bound[, max_step, ...]) | Implicit Runge-Kutta method of Radau IIA family of order 5. |
| **BDF**(fun, t0, y0, t_bound[, max_step, rtol, ...]) | Implicit method based on backward-differentiation formulas. |
| **LSODA**(fun, t0, y0, t_bound[, first_step, ...]) | Adams/BDF method with automatic stiffness detection and switching. |
| **OdeSolver**(fun, t0, y0, t_bound, vectorized) | Base class for ODE solvers. |
| **DenseOutput**(t_old, t) | Base class for local interpolant over step made by an ODE solver. |
| **OdeSolution**(ts, interpolants) | Continuous ODE solution. |

### Old API

These are the routines developed earlier for SciPy. They wrap older solvers implemented in Fortran (mostly ODEPACK). While the interface to them is not particularly convenient and certain features are missing compared to the new API, the solvers themselves are of good quality and work fast as compiled Fortran code. In some cases, it might be worth using this old API.

| | |
|---|---|
| **odeint**(func, y0, t[, args, Dfun, col_deriv, ...]) | Integrate a system of ordinary differential equations. |
| **ode**(f[, jac]) | A generic interface class to numeric integrators. |
| **complex_ode**(f[, jac]) | A wrapper of ode for complex systems. |

USING MODULE DOCUMENTATION
THE MOST USEFUL HAVE EXAMPLES AS CODE
SO YOU CAN AT LEAST UNDERSTAND THE
REQUIRED CODE STRUCTURE

OPTIONS CAN LOOK LIKE “GOBBLE-DE-GOOK”,
UNTIL YOU PRACTICE RUNNING EXAMPLES



# PART 3 - MODULES AND FUNCTIONS



- CONWAYS GAME OF LIFE
- SOME EXAMPLES
- PRACTICAL 4



chaos....

David Topping: david.topping@manchester.ac.uk

# GENERAL STRUCTURE FOR SOLVING ODES, USING SCIPY

In [ ]:

```
# Import modules
import numpy as np
from scipy.integrate import odeint

# -- define some initial conditions
# ....
# ------------------------------------
def dydt(input):
    # -- perform some calculations
    # .....
    # ------------------------------
    return dydt_array
# Define the time over which the simulation will take place
t = np.linspace(0, 10000, num =1000)
# Call the ODE solver with reference to our function , dydt ,setting the
# absolute and relative tolerance , atol and rtol respectively.
solution = odeint(dy_dt , array , t, rtol =1.0e-6, atol =1.0e-4,tcrit=None)
# Do something with the output contained within variable 'solution '
```

Import modules

↓

Define a function to solve. The 'Right Hand Side' [RHS]

↓

Define a period to provide outputs

↓

Call the ODFE solver, with relevant options

# 1) E.G. SIMULATING MULTIPLE INSTANCES OF THE RADIOACTIVE DECAY EQUATION

```
Gamma_array = np.zeros((10), dtype=float)
Gamma_array[0]=1.0
Gamma_array[1]=1.5
Gamma_array[2]=2.0
Gamma_array[3]=5.0
Gamma_array[4]=8.0
Gamma_array[5]=12.0
Gamma_array[6]=15.0
Gamma_array[7]=20.0
Gamma_array[8]=22.5
Gamma_array[9]=25.0

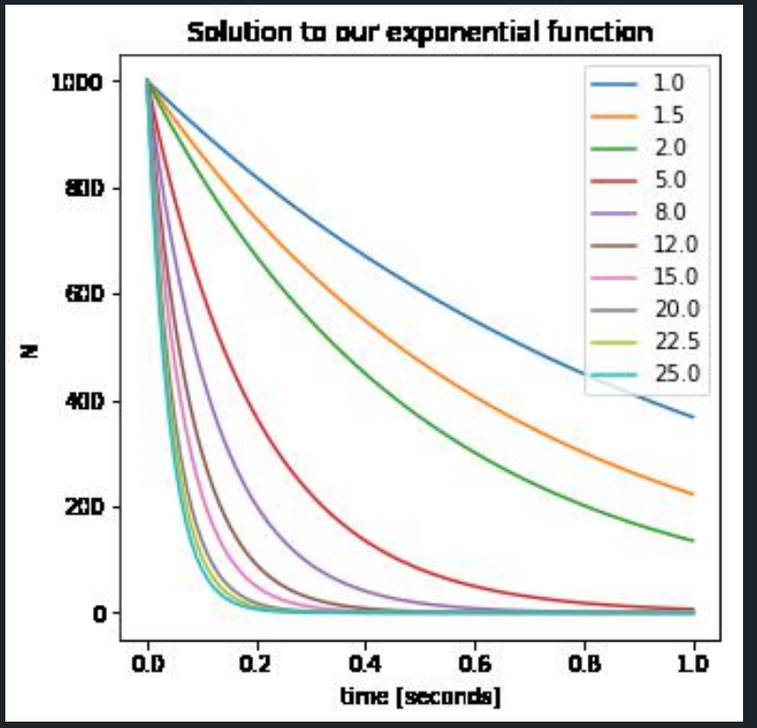
No = 1000 # Our initial value for 'N' will remain as 1000
t = np.linspace(0, 1, num=100) # time span and steps (start, end, number of entries).

# We still need a function that calculates the derivative expression
def differential_function(N, t):
    return -1.0*Gamma*N

# <<-------------------- INSERT CODE HERE ----------------------->>
# Create loop a that selects different values of the decay coefficient and calls the O
# For each solution, store these values in a new 2D array. I will create this array fo
# we know there are 100 entries in our time array and 10 decay constants
solution_store_array = np.zeros((100,10), dtype=float)

# Now define your loop. HINT, to access the contents of the 'solution' array as one co
for i in range(10):
    Gamma=Gamma_array[i]
    solution = odeint(differential_function, No, t) # integrate, refer scipy docs
    solution_store_array[:,i] = solution[:,0]
# <<------------------------------------------------------------>>
```

What do we need to import to solve this equation?



# 2) COUPLED DIFFERENTIAL EQUATIONS

e.g. The Lorentz system

$$\frac{dX}{dt} = \sigma(Y - X)$$
$$\frac{dY}{dt} = (\rho - Z)X - Y$$
$$\frac{dZ}{dt} = XY - \beta Z$$

*In popular media the 'butterfly effect' stems from the real-world implications of the Lorenz attractor, i.e. that in any physical system, in the absence of perfect knowledge of the initial conditions (even the minuscule disturbance of the air due to a butterfly flapping its wings), our ability to predict its future course will always fail.*

*The series does not form limit cycles nor does it ever reach a steady state. Instead it is an example of deterministic chaos. Two initial states no matter how close will diverge, usually sooner rather than later.*

# 2) COUPLED DIFFERENTIAL EQUATIONS - THE LORENTZ SYSTEM

```python
# import libraries we need
import numpy as np # Import Numpy as usual
from scipy.integrate import odeint # We only want to use the 'odeint' [Solve Initial Value P
import matplotlib.pyplot as plt # Import Matplotlib so we can plot results
from mpl_toolkits.mplot3d import Axes3D #Import another package that allows us to plot 'nice

# Set Lorenz paramters and initial conditions
sigma = 10
beta = 2.667
rho = 28
X0 = 5
Y0 = 1
Z0 = 1.05

# Define the differential equation function the ODE solver will use
def differential_function(array, t):
    # Notice here the function receives an array which has 3 cells
    # Outside of this function we have decided that the 1st component is X
    # .. the 2nd is Y and the 3rd is Z. Thus below we access those elements
    # using Python indexing
    X = array[0]
    Y = array[1]
    Z = array[2]
    # The Lorenz equations
    # Im using 3 new variables, with a name that hopefully makes sense.
    # These follow the equations displayed above this box
    dX_dt = sigma*(Y - X)
    dY_dt = (rho-Z)*X-Y
    dZ_dt = X*Y-beta*Z

    # We now return another array, but this contains the values for each derivative. The ord
    # has to be the same order as the variables in 'array'
    return [dX_dt, dY_dt, dZ_dt]


t = np.linspace(0, 100, num=10000) # time span and steps (start, end, number of entries). Sa

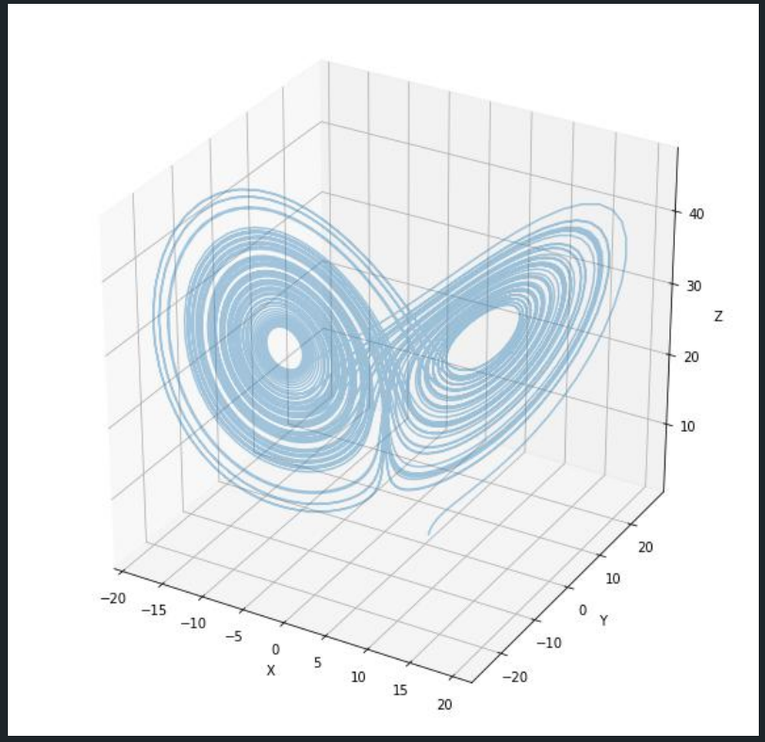
# Integrate the Lorenz equations
# Solve the differential equation using the 'odeint' package
# A couple of things to note here, again:
# 1 - The ODE solver expects key information on starting conditions and the length [duration
# 2 - The ODE solver returns an array which gives N as one column and time [t] as the other,
solution = odeint(differential_function, (X0, Y0, Z0), t)
# As we are now passing an array of variables, the function will return an array of each var
x = solution[:,0] # X is the first variable in our array, thus first column in our output
y = solution[:,1] # Y is the second variable in our array, thus second column in our output
z = solution[:,2] # Z is the third variable in our array, thus third column in our output
```

STILL DEFINE OUR ODE

STILL DEFINE A TIME PERIOD

STILL DEFINE INITIAL CONDITIONS

NOW WE PASS AN ARRAY TO THE ODE FUNCTION AND RETURN AN ARRAY

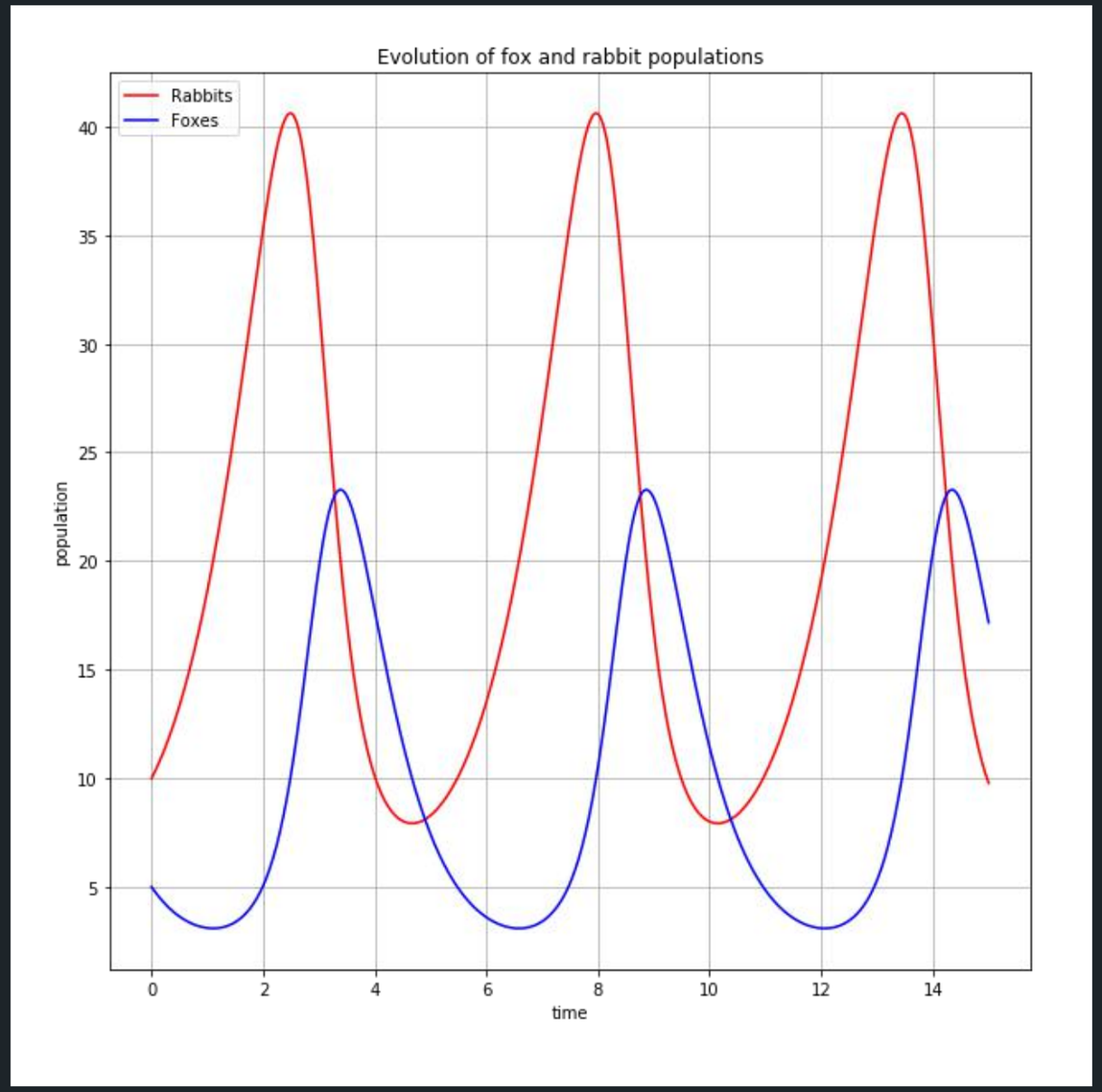


# 2) COUPLED DIFFERENTIAL EQUATIONS - THE PREDATOR-PREY EQUATIONS

$$\frac{dU}{dt} = aU - bUV$$
$$\frac{dV}{dt} = -cV + dbUV$$

Where the variables and constant parameters are defined as follows:

- U: number of preys
- V: number of predators (for example, foxes)
- a is the natural growing rate of rabbits
- b is the natural dying rate of rabbits
- c is the natural dying rate of fox
- d is the factor describing how many caught rabbits lead to an increased population of foxes.



# PART 3 - MODULES AND FUNCTIONS



- CONWAYS GAME OF LIFE
- SOME EXAMPLES
- PRACTICAL 5



chaos....

David Topping: david.topping@manchester.ac.uk

# LECTURE 5

Import modules

↓

Define a function to solve. The 'Right Hand Side' [RHS]

↓

Define a period to provide outputs

↓

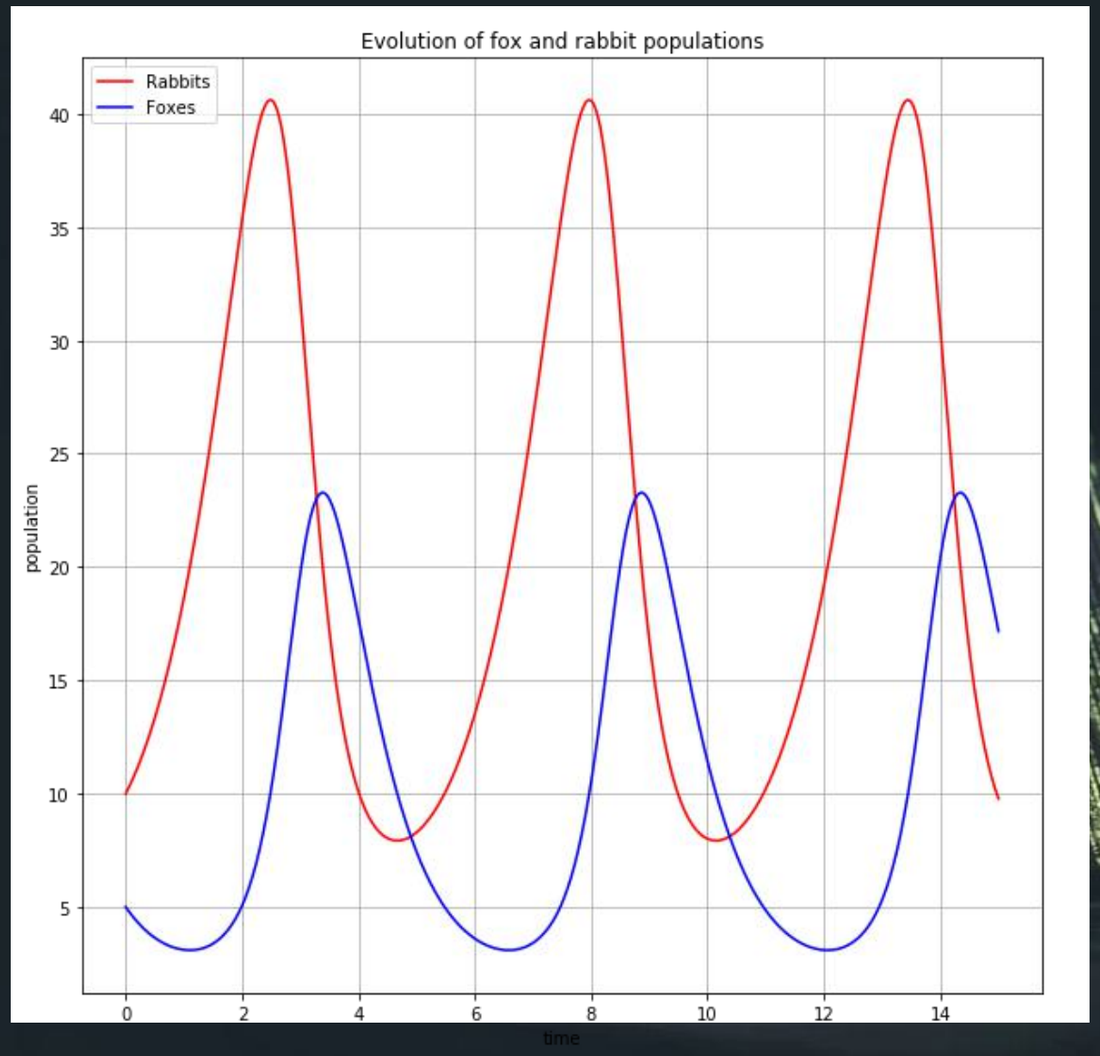
Call the ODE solver, with relevant options

# SUMMARY

## SOLVING DIFFERENTIAL EQUATIONS

- We need to integrate ordinary differential equations [ODEs]
- Where solutions to ODEs are difficult/impossible to arrive at by hand - there are functions in Python to solve/integrate
- Requires us to define the ODE to be solved through arrays and indices.

# PRACTICAL 5




# GOALS

## CODE

- Learning how we solve a single differential equation using the Scipy package
- Integrate an exponential decay equation
- Solve coupled equations: Implement the Predator-Prey equations, or the Lotka-Volterra model

## GENERAL EXPERIENCE

- Learn how to use existing modules to solve a particular problem
- Start to consider sensitivity to variables